

FORMATO PARA LAS ACTIVIDADES DE TRABAJO AUTÓNOMO, PRÁCTICAS DE APLICACIÓN Y TRABAJO COLABORATIVO

CARRERA	PLAN ESTUDIOS	DOCENTE
Desarrollo de Software	Pensum 2026	Ing. Diego Salgado.
PARCIAL	ESCENARIO	NOMBRE ASIGNATURA
2	Presencial	Metodología de Desarrollo de Software
TEMA:		De la Implementación a la Automatización de Pruebas
NOMBRE ESTUDIANTE		Hefziba Belinda Saransig Pinanjota

Introducción

La fase de implementación de software conecta el diseño con el despliegue en producción, asegurando que el sistema construido se instale, configure y ponga en funcionamiento en un entorno real de forma controlada. En esta etapa también se consolidan prácticas como la gestión de configuración, el control de versiones y la preparación de entornos donde posteriormente se ejecutarán las pruebas y la automatización.

Eje A: Implementación de software y gestión de configuración

La implementación de software es el proceso mediante el cual un sistema se instala, configura y se pone operativo en el contexto real de una organización, después de haber sido desarrollado y probado. En esta fase se planifica el despliegue, se adapta el sistema a los procesos del negocio, se migran los datos necesarios, se capacita a los usuarios y se verifica que el software cumple los requisitos funcionales y no funcionales en el entorno objetivo, actuando como puente entre lo diseñado y construido y lo que realmente se utiliza en producción. Por su impacto directo en la continuidad del servicio y en la percepción de los usuarios, se considera una etapa crítica dentro del ciclo de vida del software.

Dentro de la implementación, la gestión de configuración y el uso de sistemas de control de versiones como Git son fundamentales, ya que permiten registrar la historia de cambios del código, coordinar el trabajo de varios desarrolladores y mantener trazabilidad sobre qué versión se despliega en cada entorno. En escenarios de integración continua, el repositorio se convierte en la fuente única de verdad desde la cual servidores como Jenkins o GitHub Actions ejecutan compilaciones, pruebas y despliegues automatizados, reduciendo errores humanos y tiempos de entrega. Además, el control de versiones facilita la creación de ramas para nuevas funcionalidades, la corrección de errores en paralelo y la posibilidad de revertir rápidamente a versiones estables cuando una implementación falla, disminuyendo el riesgo durante el despliegue.

Eje B: Estrategias de pruebas unitarias e integración

Las pruebas unitarias son pruebas automatizadas de bajo nivel que se aplican sobre unidades pequeñas de código, como funciones, métodos o clases, con el objetivo de verificar que cada componente aislado se comporte exactamente como fue diseñado. Al ejecutarse desde las primeras etapas del desarrollo, permiten detectar errores de manera temprana, cuando el código todavía es reciente y poco acoplado, lo que hace que corregir defectos sea más sencillo y menos costoso que si se detectaran en fases posteriores o en producción. Una buena base de pruebas unitarias actúa como red de seguridad para la refactorización, ya que brinda confianza al introducir cambios y ayuda a evitar regresiones; además, cada test funciona como documentación ejecutable que describe el comportamiento esperado de cada componente del sistema.

Por otro lado, las pruebas de integración se centran en comprobar cómo interactúan entre sí varios módulos o servicios, evaluando que la comunicación, el intercambio de datos y los contratos entre componentes funcionen correctamente cuando se combinan. A diferencia de las unitarias, que aíslan una pieza de código, las pruebas de integración validan flujos más amplios, por ejemplo, desde una capa de servicio hasta la base de datos o la llamada a una API externa, permitiendo detectar problemas de configuración, formatos incompatibles o errores en la orquestación que no se observan al probar cada unidad por separado. En una estrategia de calidad equilibrada, ambos tipos de pruebas se consideran complementarios: las unitarias aseguran la estabilidad de cada módulo y las de integración verifican que el sistema completo se comporte de forma coherente cuando todas sus partes trabajan juntas.

Cuadro comparativo: pruebas unitarias vs. integración

Característica	Pruebas unitarias	Pruebas de integración
Objetivo	Verificar el comportamiento correcto de una unidad de código aislada (función, método, clase).	Verificar que varios módulos o servicios funcionen correctamente en conjunto y se comuniquen de forma adecuada.

Alcance	Muy limitado: una unidad o componente pequeño, con dependencias simuladas (mocks, stubs).	Más amplio: múltiples componentes, subsistemas o integraciones externas (BD, APIs, colas, etc.).
Momento típico	Durante el desarrollo, en paralelo con la codificación, ejecutadas con alta frecuencia.	Después de aprobar las pruebas unitarias, antes o durante las etapas de prueba del sistema.
Responsable habitual	Desarrollador que escribe el código y sus tests.	Equipos de QA, desarrolladores o equipos mixtos, según la estrategia de pruebas.
Impacto en mantenimiento	Reduce costos al detectar defectos temprano y respaldar refactorizaciones seguras.	Reduce riesgos de fallos en producción derivados de interacciones entre componentes.

Eje C: Automatización de pruebas y herramientas

Herramientas para pruebas unitarias

Frameworks como JUnit (Java), NUnit (.NET) y PyTest (Python) proporcionan estructuras y utilidades para definir, ejecutar y reportar pruebas unitarias de forma automatizada. Estos marcos suelen ofrecer anotaciones o decoradores para marcar métodos de prueba, aserciones para comparar resultados esperados con resultados reales y mecanismos de ejecución integrada con entornos de desarrollo o pipelines de CI.

- JUnit se ha convertido en el estándar de facto para pruebas unitarias en el ecosistema Java y fue inspiración para frameworks de otros lenguajes.
- NUnit es un framework de pruebas para .NET con sintaxis similar a JUnit, muy integrado en el ecosistema de C# y Visual Studio.
- PyTest es una herramienta flexible para pruebas en Python que soporta tanto pruebas unitarias como de integración, con una sintaxis concisa y potentes mecanismos de fixtures.

Estas herramientas resuelven el problema de ejecutar pruebas de manera repetible, automatizada y estructurada, evitando la ejecución manual de scripts ad hoc y facilitando la integración con herramientas de reporte y CI/CD.

Herramientas para pruebas de integración y de interfaz (UI)

Para pruebas de integración y de interfaz en aplicaciones web, considero que Selenium es un framework clave porque automatiza acciones de usuario en el navegador (clics, escritura, navegación) y así reduce las pruebas manuales repetitivas, aumentando la cobertura de pruebas funcionales sobre la UI. En el caso de las APIs, Postman me permite construir y organizar peticiones HTTP hacia servicios REST, definir tests sobre las respuestas y reutilizar colecciones a lo largo del ciclo de vida de la API, validando códigos de estado, estructuras de datos y reglas de negocio de forma sistemática. Por su parte, Cypress está orientado a pruebas end-to-end en aplicaciones web modernas y ejecuta los tests directamente en el navegador, ofreciendo buena visibilidad del DOM y del comportamiento del frontend, lo que facilita detectar regresiones en la interfaz y validar flujos completos que combinan frontend, backend y servicios externos, algo difícil de cubrir únicamente con pruebas unitarias.

Herramientas para automatización de flujos (CI/CD)

En cuanto a la automatización de flujos (CI/CD), Jenkins me resulta útil porque, como servidor de integración y entrega continuas, permite definir pipelines que compilan el código, ejecutan pruebas automatizadas (unitarias, de integración o E2E) y despliegan aplicaciones a distintos entornos de forma repetible cada vez que hay cambios en el repositorio. GitHub Actions, integrado en GitHub, cumple un rol similar ejecutando workflows descritos en archivos YAML en respuesta a eventos como pushes o pull requests, de modo que es posible compilar, probar y desplegar sin intervención manual directamente desde el repositorio. En mi opinión, ambas herramientas resuelven la necesidad de orquestar automáticamente tareas de compilación, prueba y despliegue, reduciendo errores humanos y el tiempo entre un cambio de código y su llegada a producción, sobre todo cuando se integran con frameworks de pruebas como JUnit, PyTest, Selenium o Cypress dentro de pipelines de CI/CD.

Herramienta	¿Qué problema resuelve? (mi opinión)	¿A qué lenguaje o entorno está orientada?	¿Licencia / tipo?
JUnit	Para mí, JUnit permite comprobar de forma rápida y repetible si mi código Java hace lo que espero, sin ejecutar la aplicación manualmente cada vez, lo que me ayuda a detectar errores pequeños en métodos y clases antes de llegar a producción.	Entiendo que está pensada principalmente para proyectos en Java y, en general, para aplicaciones sobre la JVM , integrándose bien con IDEs y herramientas como Maven o Gradle.	Según lo que revisé, es de código abierto (Open Source) y se puede usar gratis en proyectos académicos y profesionales.
NUnit	Yo percibo que NUnit resuelve la necesidad de tener un marco similar a JUnit pero para .NET, permitiéndome validar la lógica de negocio sin probar toda la aplicación a mano en cada cambio.	Identifico que está orientada al entorno .NET , sobre todo a proyectos en C# , e integra bien con Visual Studio y el ecosistema Microsoft.	A partir de mi investigación, concluyo que también es de código abierto (Open Source) y de uso gratuito.
PyTest	Yo opino que PyTest me facilita escribir y ejecutar pruebas en Python de forma sencilla y legible, sin tener que crear scripts manuales para probar cada función; esto me da más confianza en el comportamiento de mi código.	Reconozco que está claramente orientada al lenguaje Python y se usa tanto en proyectos pequeños como en aplicaciones de backend o ciencia de datos.	De acuerdo con lo que revisé, PyTest es de código abierto (Open Source) y tiene una comunidad activa que aporta extensiones.
Selenium	Mi interpretación es que Selenium me evita probar una página web de forma manual cada vez que cambio algo, porque me permite automatizar clics, navegación y entradas en el navegador como si fuera un usuario real, reduciendo el tiempo en pruebas repetitivas.	Según mi análisis, no se limita a un solo lenguaje: soporta Java , C# , Python , JavaScript , Ruby , entre otros, siempre orientado a automatizar navegadores como Chrome o Firefox.	Concluyo que Selenium es un proyecto de código abierto (Open Source) , muy usado como base de marcos de pruebas automatizadas.
Postman	Yo creo que Postman resuelve el problema de probar APIs de forma improvisada, ofreciéndome una herramienta gráfica donde defino peticiones HTTP,	Comprendo que no está ligado a un lenguaje concreto, sino al entorno de pruebas de APIs HTTP/REST , por lo que puedo usarlo con	Según mi revisión, tiene una versión gratuita muy completa, pero es un producto propietario con planes

	guardo colecciones y reviso fácilmente las respuestas, lo que hace más ordenado y rápido probar y depurar servicios REST.	servicios desarrollados en cualquier lenguaje mientras expongan endpoints.	comerciales que añaden funciones avanzadas.
Cypress	Yo considero que Cypress me ayuda a hacer pruebas end-to-end de aplicaciones web modernas de forma muy cercana al navegador, con buena visibilidad de lo que pasa en el frontend, lo que me facilita entender por qué falla algo en la interfaz y reproducir flujos de usuario reales.	Identifico que está orientada al ecosistema JavaScript/TypeScript y se usa para probar aplicaciones web en el navegador (React, Angular, Vue, etc.).	Tras investigar, concluyo que el runner de pruebas es open source , mientras que la plataforma en la nube y servicios adicionales se ofrecen bajo modelos comerciales .
Jenkins	Yo opino que Jenkins me evita ejecutar manualmente tareas repetitivas como compilar, probar y desplegar la aplicación en cada cambio; al definir pipelines automatizados puedo asegurar que cada commit pasa por un proceso estándar de calidad.	Entiendo que está desarrollado en Java , pero se usa como servidor de automatización para proyectos casi en cualquier lenguaje, gracias a sus plugins y a la ejecución de distintos tipos de scripts.	Según mi investigación, Jenkins es de código abierto (Open Source) y tiene una comunidad amplia que mantiene y extiende la herramienta.
GitHub Actions	Yo considero que GitHub Actions resuelve el problema de montar servidores externos para CI/CD, porque me permite ejecutar compilaciones, pruebas y despliegues directamente desde GitHub cuando ocurre un evento en el repositorio, acercando la automatización al lugar donde vive el código.	Observo que está orientada al ecosistema de repositorios GitHub , pero soporta proyectos en cualquier lenguaje mediante runners de Linux, Windows o macOS definidos en archivos YAML.	Según lo que revisé, forma parte del servicio GitHub , con uso gratuito limitado por minutos y almacenamiento y planes comerciales ; muchas acciones de la comunidad sí son open source .

Referencias:

- Tahchiev, P., Leme, F., Massol, V., & Gregory, G. (2010). *JUnit in action*. Manning Publications Co..
- Tudose, C. (2020). *JUnit in action*. Manning.
- Hamilton, B. (2004). *NUnit Pocket Reference: Up and Running with NUnit*. " O'Reilly Media, Inc."
- Jayaraman, K. D., & Shrivastav, E. A. (2025). Automated testing frameworks: a case study using selenium and NUnit. *International Journal of Research in Humanities & Social Sciences*, 13(1), 1-16.
- Okken, B. (2022). *Python testing with Pytest: simple, rapid, effective, and scalable*. The Pragmatic Programmers LLC.
- Speth, S., & Pretschner, A. (2025, April). GeMTest: A general metamorphic testing framework. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)* (pp. 41-44). IEEE.
- Thooriqoh, H. A., Annisa, T. N., & Yuhana, U. L. (2021). Selenium framework for web automation testing: A systematic literature review. *JUTI: Jurnal Ilmiah Teknologi Informasi*, 65-76.
- Jin, J., Fang, Z., Chen, L., & Gao, Y. (2025). PostMan: A Productive System for Spatio-temporal Data Management and Analysis: J. Jin et al. *Data Science and Engineering*, 1-24.
- Mobaraya, F., & Ali, S. (2019, December). Technical analysis of selenium and cypress as functional automation framework for modern web application testing. In *9th international conference on computer science*. sn.
- Decan, A., Mens, T., Mazrae, P. R., & Golzadeh, M. (2022, October). On the use of github actions in software development repositories. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)* (pp. 235-245). IEEE.